

Designing a DevSecOps Toolchain to Support ETL Workflows in Regulated Industries

Daniel Dickerson

October 6, 2025

Contents

1	Introduction	4
1.1	Context and Motivation	4
1.2	Problem Statement	4
1.3	Research Objectives	4
1.4	Contributions	4
1.5	Thesis Structure	4
2	Literature Review	5
2.1	Background	5
2.2	Defining Key Concepts	5
2.2.1	DevSecOps	5
2.2.2	ETL	6
2.2.3	Auditability	6
2.2.4	Governance	6
2.2.5	SDLC	6
2.2.6	Data Life Cycle	6
2.3	Research Questions	6
2.3.1	Main Research Question	6
2.3.2	Sub-questions	7
2.4	RQ1: ETL Pipelines in Regulated Industries	7
2.5	RQ2: DevSecOps and Governance in the SDLC	7
2.6	RQ3: Comparing the Data Lifecycle and the SDLC	7
2.7	RQ4: Tooling for Auditability and Governance	7
2.8	Synthesis	7
3	Methodology	8
3.1	Research Approach	8
3.2	Design Science Process	8
3.3	Case Study Context	8
3.4	Artifact Design Process	8
3.5	Evaluation Plan	8
3.6	Limitations	8
4	Baseline Pipeline	9
4.1	Overview	9
4.2	Architecture	9
4.3	Functionality	9
4.4	Observed Gaps	9
4.5	Summary	9
5	Proposed Architecture	10
5.1	Design Principles	10
5.2	High-Level Overview	10
5.3	Core Components	10
5.4	Integration with DevSecOps Practices	10
5.5	Expected Outcomes	10

6	Prototype Implementation	11
6.1	Implementation Environment	11
6.2	Toolchain Configuration	11
6.3	Security and Governance Layers	11
6.4	Demonstration	11
6.5	Challenges	11
7	Evaluation	12
7.1	Evaluation Framework	12
7.2	Experimental Setup	12
7.3	Results	12
7.4	Discussion	12
7.5	Threats to Validity	12
7.6	Summary	12
8	Discussion	13
8.1	Summary of Findings	13
8.2	Implications for Practice	13
8.3	Limitations	13
8.4	Future Work	13
9	Conclusion	14
9.1	Restating the Research Question	14
9.2	Key Contributions	14
9.3	Reflections	14
9.4	Closing Remarks	14

Chapter 1

Introduction

1.1 Context and Motivation

Explain why ETL workflows in regulated industries matter. Discuss challenges of compliance, auditability, and governance in data pipelines.

1.2 Problem Statement

Clearly state the problem: lack of integrated DevSecOps governance and auditability in ETL workflows.

1.3 Research Objectives

Summarize the purpose and aims of the study. List how your design and prototype address the identified gap.

1.4 Contributions

List the expected outcomes of this thesis:

- Architectural model of a DevSecOps-enabled ETL toolchain
- Prototype implementation
- Evaluation demonstrating improved auditability and governance

1.5 Thesis Structure

Briefly describe each chapter's purpose and contents.

Chapter 2

Literature Review

This chapter reviews the state of knowledge relevant to the design of ETL pipelines in regulated industries and the application of DevSecOps principles. It begins by introducing the context of ETL and governance requirements, then defines the key concepts used in this thesis. From this foundation, the research questions are posed explicitly. Each subsequent subsection addresses one sub-question through a focused review of the literature.

2.1 Background

- pipelines underpin data-intensive systems, producing usable data for fraud detection, market analysis, compliance reporting, and application metadata processing.

- Although the extract-transform-load process appears straightforward, in practice it is resource-intensive, complex, and often the most time-consuming part of data warehousing [1].

- ETL pipelines function as the “eyes of the system,” feeding downstream analytics and operations; failures or opacity here undermine the reliability of the entire enterprise stack.

- In regulated industries, compliance is tied as much to *how* data is processed as to the outcomes produced.

- Regulatory frameworks (e.g., GDPR, SOX, APRA) require secure and responsible handling of data, not just correct outputs.

- Auditability can be understood as the ability to demonstrate compliance through reproducibility, lineage, and tamper-resistant logs. [Find relevant citation here]

- Governance encompasses the policies, frameworks, and controls that institutionalize compliance practices. [Find relevant citation here]

- Failures in either area expose organizations to fines, license revocation, reputational damage, and loss of investor trust. [Find relevant citation here]

- DevOps was originally conceived to accelerate software delivery cycles through automation,

CI/CD, and rapid feedback loops. [Find relevant citation here]

- DevSecOps extends this model by embedding security and compliance throughout the software development lifecycle, using practices such as automated testing, compliance-as-code, artifact signing, and immutable logging [2, 3].

- These practices generate auditable artifacts that can be inspected by regulators and governance stakeholders, bridging engineering and compliance concerns.

- Data itself is increasingly recognized as a valuable artifact, often outweighing the systems that process it. [Find relevant citation here]

- This shift has intensified regulatory scrutiny of data workflows, highlighting the need to extend DevSecOps principles into ETL pipelines.

2.2 Defining Key Concepts

2.2.1 DevSecOps

DevSecOps can be understood as the proactive and continuous enforcement of security and compliance requirements throughout the software development lifecycle (SDLC) [2]. Opposed to approaches that treat security as a separate stage or external review, DevSecOps seeks to embed security within every stage of the lifecycle [4]. In this model, compliance is the default state, and deviations from established requirements are treated as explicit exceptions that must be justified [3].

A defining characteristic of DevSecOps is its reliance on automation to enforce non-functional requirements; such as auditability and governance. This is achieved through testing, provisioning, monitoring, logging, and policy enforcement [2, 5]. These automated controls produce auditable evidence, such as immutable logs and signed artifacts, that can be evaluated by regulators or auditors; effectively bridging the concerns of development teams and governance stakeholders [4].

Through the integration of cultural expectations and shared responsibility with technical mech-

anisms for enforcement, DevSecOps establishes a framework where security and compliance are continuous, measurable, and intrinsic to software delivery, rather than externally imposed constraints.

2.2.2 ETL

Extract-Transform-Load (ETL) is a structured pattern of processes, that ingests data, transforms it into analytics-ready formats, and persists it into target repositories such as data warehouses, data lakes, or file stores [1]. ETL is traditionally characterized by three interdependent stages: extraction, the sourcing of data from diverse and often inconsistent operational systems; transformation, the cleansing, standardization, integration, and reshaping of this data into consistent semantic structures; and loading, the deposition of transformed data into persistent storage for later use [1]. Kimball and Caserta emphasize that ETL has historically represented the most time-consuming and resource-intensive component of data warehousing, serving as the critical link between operational data and use.

While the classical framing describes ETL as a sequential and batch-oriented workflow, modern practice reflects more flexible execution models, including extract-load-transform (ELT) and hybrid streaming-batch architectures that exploit distributed and cloud-native infrastructure [6]. Beyond data movement, ETL operationalizes data integration, ensures quality and standardization, and increasingly serves as a locus for governance and compliance requirements. In regulated industries, ETL pipelines are expected not only to provide scalable data preparation, but also to guarantee lineage, reproducibility, and auditability as part of broader data governance frameworks [6].

Accordingly, ETL should not be conceived as a rigid boundary of operations, but as a flexible socio-technical pattern that concludes once data is made persistent and reusable, yet intersects with wider lifecycle concerns such as orchestration, monitoring, and policy enforcement. In this sense, ETL pipelines underpin both analytic scalability and the assurance of institutional trust and regulatory legitimacy [6].

2.2.3 Auditability

Auditability is a designed property of systems that enables their processes, controls, and outcomes to be rendered knowable and verifiable by stakeholders. Weigand and Elsas [7] define auditability through five interdependent requirements: *normativity*, the presence of a clear regulatory or contractual framework governing operations; *accountabil-*

ity, the production of verifiable statements linking actions to responsible agents and norms; *referentiality infrastructure*, the existence of records that connect operations to points of first recording; *reliability*, the faithful and tamper-resistant representation of events; and *verifiability*, the ability of external parties to test or validate the correctness of claims. Although this originates in the domain of financial reporting and organizational control, it provides a transferable structure for information systems more broadly.

Extending these principles into technical settings, Wang et al. [8] demonstrate how auditability can be operationalized for cloud storage through mechanisms such as third-party verification, cryptographic proofs of integrity, and support for dynamic data operations. Their work highlights that auditability is not merely an organisational concept, but one that can be concretely engineered into data management infrastructures.

In the context of ETL pipelines and DevSecOps practices, these requirements translate directly into regulated-industry concerns. *Normativity* implies embedding compliance and data protection policies within workflow execution; *accountability* requires associating each transformation or deployment with authenticated agents or automated processes; *referentiality infrastructure* corresponds to full lineage tracking across extraction, transformation, and loading; *reliability* is realised through immutable logs, signed artifacts, and cryptographic hashing; and *verifiability* involves reproducibility testing and the possibility of independent or delegated audits via CI/CD-integrated checks. Taken together, these adapted requirements frame auditability in ETL pipelines as a designed capability: every data operation and system change can be traced, reproduced, and independently validated against regulatory expectations.

2.2.4 Governance

2.2.5 SDLC

2.2.6 Data Life Cycle

2.3 Research Questions

Drawing from the gaps identified in prior work, this paper will address the following:

2.3.1 Main Research Question

How can a DevSecOps toolchain be designed to support ETL workflows in regulated industries, enhancing auditability and governance?

2.3.2 Sub-questions

1. How does ETL manifest in regulated industries, and what auditability and governance challenges are faced?
2. How do DevSecOps practices enhance auditability and governance within the software development lifecycle?
3. In what ways do the stages of the data lifecycle align with or diverge from the stages of the software development lifecycle, and how does this affect the integration of DevSecOps practices?
4. What are the key categories of tooling in DevSecOps and in ETL pipelines, and how do their features compare in terms of supporting auditability and governance?

2.4 RQ1: ETL Pipelines in Regulated Industries

2.5 RQ2: DevSecOps and Governance in the SDLC

2.6 RQ3: Comparing the Data Lifecycle and the SDLC

2.7 RQ4: Tooling for Auditability and Governance

2.8 Synthesis

Chapter 3

Methodology

3.1 Research Approach

Explain the overall design science methodology used. Justify its selection for solving an engineering problem through artifact design.

3.2 Design Science Process

Describe each stage:

- **Problem identification:** Auditability and governance gaps in ETL.
- **Artifact design:** A DevSecOps toolchain architecture.
- **Demonstration:** Implementing the prototype.
- **Evaluation:** Measuring lineage, reproducibility, and tamper detection.

3.3 Case Study Context

Briefly describe the regulated domain (e.g., financial systems, SEC filings dataset). Justify its representativeness and constraints.

3.4 Artifact Design Process

Detail the iterative development process—tools, versioning, and validation loops.

3.5 Evaluation Plan

Summarize how you'll measure auditability, reproducibility, and tamper resistance.

3.6 Limitations

Acknowledge what is out of scope or constrained (e.g., scalability, dataset variety).

Chapter 4

Baseline Pipeline

4.1 Overview

Describe the initial ETL setup (Airflow + Spark + S3, deployed via Docker). Explain its purpose as a comparison point.

4.2 Architecture

Insert a diagram showing the baseline system structure. Highlight key components and data flow.

4.3 Functionality

Detail how data moves through extraction, transformation, and loading.

4.4 Observed Gaps

Discuss deficiencies in:

- Lineage tracking
- Reproducibility
- Governance and security controls

4.5 Summary

Conclude with why the baseline cannot meet auditability expectations.

Chapter 5

Proposed Architecture

5.1 Design Principles

Outline the guiding principles of the proposed DevSecOps-integrated ETL system: automation, immutability, traceability, reproducibility.

5.2 High-Level Overview

Provide an architectural diagram comparing baseline and proposed design.

5.3 Core Components

Explain each layer:

- **Infrastructure as Code:** Terraform for provisioning.
- **Orchestration:** Airflow for controlled execution.
- **Security:** CI/CD checks, artifact signing, SAST/SCA.
- **Governance:** Immutable audit logs, metadata tracking.

5.4 Integration with DevSecOps Practices

Map the proposed pipeline to the DevSecOps lifecycle (plan → code → build → test → release → monitor).

5.5 Expected Outcomes

Summarize the architectural advantages for auditability and compliance.

Chapter 6

Prototype Implementation

6.1 Implementation Environment

List the environment setup: AWS EMR, EC2, S3, Docker, GitHub Actions, Terraform.

6.2 Toolchain Configuration

Show how tools interconnect (e.g., Airflow triggers Spark, CI/CD automates IaC).

6.3 Security and Governance Layers

Describe:

- Hash signing of artifacts
- Immutable logging mechanisms
- Versioned configuration and audit history

6.4 Demonstration

Summarize execution flow with example runs and screenshots.

6.5 Challenges

Briefly discuss debugging, integration issues, and trade-offs during build.

Chapter 7

Evaluation

7.1 Evaluation Framework

Define how the artifact is assessed — e.g., lineage completeness, reproducibility, tamper detection.

7.2 Experimental Setup

Describe datasets, test environments, and CI/CD validation routines.

7.3 Results

Present tables, metrics, and visualizations of outcomes.

7.4 Discussion

Interpret results in context: - How did the toolchain improve governance? - What measurable gains were observed?

7.5 Threats to Validity

Discuss internal and external validity considerations.

7.6 Summary

Wrap up the findings and prepare for the discussion/conclusion chapters.

Chapter 8

Discussion

8.1 Summary of Findings

Recap the main evaluation results and architectural improvements.

8.2 Implications for Practice

Explain how this approach could generalize to other regulated environments.

8.3 Limitations

Note scalability, data type constraints, or untested regulatory frameworks.

8.4 Future Work

Propose enhancements—e.g., integrating ML pipelines, policy-as-code frameworks, or extended logging.

Chapter 9

Conclusion

9.1 Restating the Research Question

Reiterate the main question and summarize how your design addressed it.

9.2 Key Contributions

List concrete deliverables (architecture, prototype, metrics, evaluation).

9.3 Reflections

Discuss broader lessons about DevSecOps integration into data pipelines.

9.4 Closing Remarks

Tie back to the motivation and note the continuing relevance for regulated industries.

Bibliography

- [1] R. Kimball and J. Caserta, *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Wiley, 2004, iISBN: 9780764567575. [Online]. Available: <https://www.oreilly.com/library/view/the-data-warehouse/9780764567575/>
- [2] M. Anisetti, N. Bena, F. Berto, and G. Jeon, “A DevSecOps-based Assurance Process for Big Data Analytics,” in *2022 IEEE International Conference on Web Services (ICWS)*, Jul. 2022, pp. 1–10. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9885738>
- [3] C. Castellanos, C. A. Varela, and D. Correal, “ACCORDANT: A domain specific-model and DevOps approach for big data analytics architectures,” *Journal of Systems and Software*, vol. 172, p. 110869, Feb. 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121220302594>
- [4] M. Anisetti, C. A. Ardagna, E. Damiani, and F. Gaudenzi, “A Semi-Automatic and Trustworthy Scheme for Continuous Cloud Service Certification,” *IEEE Transactions on Services Computing*, vol. 13, no. 1, pp. 30–43, Jan. 2020. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7831357/authors>
- [5] Y. Zhou, Y. Yu, and B. Ding, “Towards MLOps: A Case Study of ML Pipeline Platform,” in *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, Oct. 2020, pp. 494–500. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9361315>
- [6] D. Mahmud and M. Z. Ikbali, “THE ROLE OF ETL (EXTRACT-TRANSFORM-LOAD) PIPELINES IN SCALABLE BUSINESS INTELLIGENCE: A COMPARATIVE STUDY OF DATA INTEGRATION TOOLS,” *ASRC Procedia: Global Perspectives in Science and Scholarship*, vol. 2, no. 1, pp. 89–121, Apr. 2022. [Online]. Available: <https://global.asrcconference.com/index.php/asrc/article/view/29>
- [7] H. Weigand and P. Elsas, “Auditability as a Design Problem,” in *2019 IEEE 21st Conference on Business Informatics (CBI)*, vol. 01, Jul. 2019, pp. 276–285, iISSN: 2378-1971. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8808092>
- [8] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, “Enabling Public Auditability and Data Dynamics for Storage Security in Cloud Computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 22, no. 5, pp. 847–859, May 2011. [Online]. Available: <https://ieeexplore.ieee.org/document/5611497>